

Investigation Report: Calendar Email Notification System

Summary

The calendar scheduling system (`/agendamentos`) has a complete email notification implementation that sends confirmation emails with Google Calendar links when equipment is booked. **The code is correct**, but there is **one critical deployment issue** that is almost certainly the root cause of emails not being sent in production.

Files Involved

File	Role
<code>lib/email/agendamento-email.ts</code>	Core email logic — Gmail SMTP via nodemailer, HTML template, Google Calendar link
<code>app/api/agendamentos/route.ts</code>	POST handler creates booking + fires email (fire-and-forget)
<code>app/api/agendamentos/[id]/route.ts</code>	GET/DELETE for individual bookings (no email on delete)
<code>app/api/cron/cleanup-agendamentos/route.ts</code>	Cron job to delete past bookings (no email involvement)
<code>app/agendamentos/page.tsx</code>	Frontend calendar UI — calls <code>POST /api/agendamentos</code>
<code>app/agendamentos/settings/page.tsx</code>	Admin equipment/authorization management
<code>app/api/agendamento/route.ts</code>	Legacy redirect to <code>/api/agendamentos</code>

Email Flow (How It Works)

```

User clicks "Book" in /agendamentos
↓
Frontend calls POST /api/agendamentos
↓
API creates booking in DB (prisma.agendamento.create)
↓
API calls sendAgendamentoEmails() – FIRE AND FORGET (no await)
↓
API returns 201 immediately (user sees success)
↓ (background)
sendAgendamentoEmails() creates Gmail SMTP transporter
↓
Collects recipients: creator, target user, equipment managers, advisor (if external)
↓
Generates Google Calendar link from raw dates
↓
Sends HTML email to each recipient (10s timeout each, 15s batch timeout)
↓
Closes transporter connection

```



ROOT CAUSE: EMAIL_PASS Not Configured on Render

The Problem

The `render.yaml` declares these environment variables:

```

envVars:
- key: DATABASE_URL
- key: NEXTAUTH_SECRET
- key: NEXTAUTH_URL
- key: GOOGLE_CLIENT_ID
- key: GOOGLE_CLIENT_SECRET
- key: NODE_ENV

```

EMAIL_USER and EMAIL_PASS are NOT listed. This means on Render:

1. `process.env.EMAIL_PASS` is `undefined` (empty string)
2. The `createTransporter()` function checks `if (!pass)` on line 29-31
3. It logs: `[Email] EMAIL_PASS not configured – emails will be skipped`
4. Returns `null` instead of a transporter
5. `sendAgendamentoEmails()` sees `null` transporter and skips all emails silently

Evidence

- `.env.local` (local only, gitignored) has the correct credentials
- `.env.example` documents the variables but doesn't configure them
- SMTP test with the credentials **works perfectly** from this machine
- The code itself is well-implemented with proper error handling

The Fix

Add these two environment variables to Render:

Key	Value
EMAIL_USER	lerpfeq@gmail.com
EMAIL_PASS	kwug mrfv gkxg xzhp

How to add on Render:

1. Go to <https://dashboard.render.com>
2. Select the `lerp-intranet` service
3. Go to **Environment** tab
4. Click **Add Environment Variable**
5. Add `EMAIL_USER` = `lerpfeq@gmail.com`
6. Add `EMAIL_PASS` = `kwug mrfv gkxg xzhp`
7. Click **Save Changes**
8. Render will auto-restart the service

! Secondary Concern: Fire-and-Forget on Serverless

Current Pattern (line 213 of `route.ts`):

```
// Fire-and-forget: NO await – response returns immediately
sendAgendamentoToEmails(emailPayload, responsavelEmails, isExterno)
  .then(() => console.log('[Agendamento] ✅ Email delivery completed in background'))
  .catch((err) => console.error('[Agendamento] ❌ Email delivery failed in background:', err));
```

Why This Matters

- On **Render Web Service** (`next start`): This works fine because it's a persistent Node.js process
- On **serverless/edge runtimes**: The function would be killed before emails finish sending
- Since Render runs `npm start` (persistent server), this is **NOT a problem currently**
- But if ever migrated to Vercel Serverless, this would need `waitUntil()` or similar

Verdict: ✅ OK for current Render deployment

Code Quality Assessment

Aspect	Status	Notes
SMTP Configuration	✓ Good	Gmail service, App Password, proper timeouts
Error Handling	✓ Good	Individual per-recipient try/catch, doesn't throw
Timeout Protection	✓ Good	10s per email, 15s batch, connection/socket timeouts
Resource Cleanup	✓ Good	<code>transporter.close()</code> in finally block
Recipient Logic	✓ Good	Creator + target + managers + advisor (if external)
HTML Template	✓ Good	Professional, includes Google Calendar link
Fire-and-Forget	✓ OK	Doesn't block API response, logs results
Graceful Degradation	✓ Good	Missing <code>EMAIL_PASS</code> → skips silently (no crash)

Optional Improvements

1. **Update** `render.yaml` to document `EMAIL_USER` and `EMAIL_PASS` :

```
yaml
- key: EMAIL_USER
  sync: false
- key: EMAIL_PASS
  sync: false
```

2. **Add email on booking deletion** (currently no notification when cancelled)

3. **Add a test email endpoint** for admin to verify email works:

```
POST /api/admin/test-email → sends test to admin's own email
```

Cron Job

There's a cleanup cron at `GET /api/cron/cleanup-agendamentos?token=lerp-cron-2026` that deletes past bookings. This has **nothing to do with email** — it's just database cleanup. No issues found here.

Conclusion

Issue	Root Cause	Fix
Emails not sent in production	<code>EMAIL_PASS</code> env var missing on Render	Add it in Render dashboard
Code bugs	None found	N/A
Architecture issues	None for current deployment	N/A

The **ONLY** action needed is adding `EMAIL_USER` and `EMAIL_PASS` to Render's environment variables.